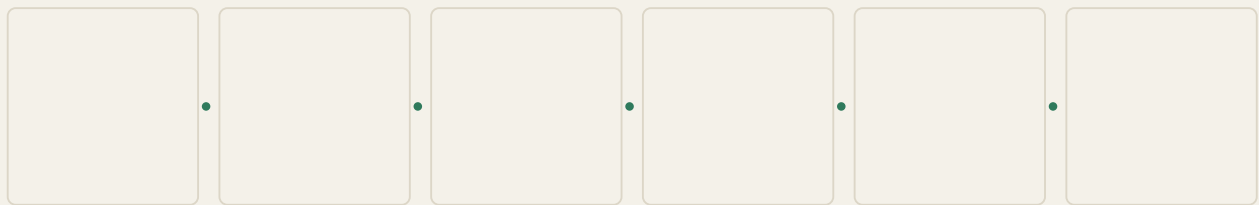


Preferred Drug Assignment

How an AI pipeline decides which covered drugs a health plan should recommend first

A complete, plain-language guide to the high level design. Written for readers who have never seen a formulary, an embedding, or a vector database.

Traced end to end through three real drugs: a single-ingredient insulin, a two-ingredient antibiotic, and a shingles vaccine.



Who this guide is for

This guide explains one production system from top to bottom. It assumes no pharmacy knowledge and no machine learning knowledge. Every term is defined where it first appears.

Reader	Where to start
Recruiters and hiring managers	Chapters 1, 2 and 18 give the problem, the journey of one drug, and the engineering lessons, with no technical prerequisites.
Pharmacists and clinical reviewers	Chapters 11, 13 and 15 cover the matching rules, the safety controls, and exactly what a human reviewer is asked to check.
Product managers	Chapters 1, 7, 15 and 16 cover the business case, the coverage of all seven plans, the review artifact, and what changes in the live database.
Engineers	Chapters 4, 8, 9, 10, 12, 13 and 16 cover extraction, the two data stores, similarity filtering, the evidence packet, retries, and reconciliation.

How to read this guide

The chapters follow the data, not the code. Each one opens with why the step exists, then shows what goes in and what comes out. Coloured boxes flag the six things worth slowing down for:

Callout	Meaning
Plain-language definition	A term is being introduced for the first time.
Why it matters	The business or clinical reason a step exists.
Input and Output	The concrete data entering or leaving a stage.
Safety control	A deterministic check that constrains the AI.
Human review	A point where a person inspects the result.
Limitation	Something the current implementation does not do.

NOTE

This guide separates observed values from illustrative ones. Numbers drawn from the source list, the code, or the Medicare catalogue snapshot are labelled as observed. Similarity scores and any figure that would require a full production run are labelled illustrative, and are there to show the shape of the data rather than to report a measurement.

Contents

1	The Business Problem	What a preferred drug list is and why matching is hard
2	The Journey of Three Drugs	One insulin, one combination antibiotic, one vaccine
3	Read the Source List	The messy one-column input
4	Read the Drug Name and Class	Constrained extraction with GPT-4o
5	Merge Doses of the Same Drug	Grouping product rows into drug records
6	One Row Per Drug	The checkpoint contract
7	Search All Seven Formularies	Fan-out retrieval across plans
8	The Paired Data Stores	Vector search alongside the record database
9	Keep the Closest Matches	Score tiers rather than top rows
10	Build the Closed Evidence Packet	Bounding what the model can see
11	Rules for Same Drug	The clinical matching protocol
12	AI Clinical Judgment	What the model decides and what it does not
13	Block Unsupported Matches	The intersection guard
14	Resolve Real Drug Names	Turning identifiers back into names
15	Pharmacist Review	The report and its four checks
16	Clear Old Flags and Set New Ones	Desired-state reconciliation

17 The Live Formulary Record

What actually changes

18 End to End Summary

Five architectural lessons

A. Glossary

B. The extraction prompt

C. The matching prompt

D. Data counts and snapshot notes

E. Known limitations

F. Source references

The pipeline at a glance

Six stages. The first three clean up the list of drugs the plan wants to recommend. The middle two find and confirm the matching records in the plan's own catalogues. The last one writes the result to the live database.

Stage		What happens	Kind
01	Read the source list	1,355 free-text lines written by pharmacy staff	Observed
02	Read the drug name and class	A language model turns each line into a clean name plus a therapeutic class	AI
03	Merge doses of the same drug	Rows that differ only by strength collapse into one drug record	Code
04	Search all seven formularies	Each drug record is searched against every plan catalogue at the same time	Search
05	Decide which are the same drug	A language model judges the shortlist, then code discards anything it invented	AI + Code
06	Clear old flags and set new ones	The live database is reconciled to the verified result	Code

Figure 0.1 The six stages. Kind indicates whether the step is a language model decision, deterministic code, or a search service.

WHY IT MATTERS

Two of the six stages use a language model. The other four are ordinary code. That split is deliberate and is the central design idea of this system: the model is used only where judgment is genuinely needed, and its output is never trusted without a deterministic check behind it.

CHAPTER 1

The Business Problem

Two lists of drugs, written by different people for different purposes, that have to be reconciled without a shared identifier.

What a formulary is

PLAIN-LANGUAGE DEFINITION

Formulary. The list of medicines a health insurance plan agrees to pay for. If a drug is on the formulary it is covered. If it is not, the patient generally pays the full price. Each insurance plan has its own formulary, and the plan studied here runs seven of them.

A formulary is not a short list. The Medicare catalogue examined for this guide holds 4,579 individual product records covering 1,795 distinct drug names. A doctor prescribing within that plan is choosing from thousands of covered options.

What a preferred drug list is

PLAIN-LANGUAGE DEFINITION

Preferred Drug List, or PDL. A shorter list, written by the plan's clinical committee, of the drugs it would like prescribers to reach for first. Preferred usually means a lower copayment for the patient and fewer authorisation hurdles for the prescriber. It is a clinical and cost recommendation layered on top of coverage.

The source list studied here contains 1,355 entries. It arrives as a single column of free text in a spreadsheet, written by pharmacists for pharmacists.

Why matching the two is hard

Both lists describe medicines, but they were written independently and share no common identifier. Nothing in the preferred list points at a record in the formulary. The names themselves are the only bridge, and the names do not agree.

The obstacle	What it looks like in the real data
The same drug, named differently	The preferred list says insulin aspart (NOVOLOG) injection VIAL 100 unit/mL (10 mL). The Medicare catalogue stores Insulin Aspart and, separately, NovoLOG. One human concept, several database rows, no shared key.
One drug, many rows	Sixteen of the 1,355 preferred entries are insulin aspart. They differ only by brand, container, and concentration. Searching sixteen times for what is clinically one decision wastes effort and invites inconsistent answers.
Brand names that are absent	The preferred list names BACTRIM and SEPTRA. Neither string appears anywhere in the Medicare catalogue, which files the drug under its two generic ingredients. A text search for the brand returns nothing.
Therapeutic classes that disagree	The catalogue files Shingrix under the class Vaccines but files Zostavax, also a shingles vaccine, under Non-FRF Drugs. Matching on class alone would put these two in different worlds.

The obstacle	What it looks like in the real data
Devices mixed in with medicines	Lancets, pen needles, and glucose meters sit in the same source column as tablets and injections. They need entirely different matching logic.

Table 1.1 Five reasons a simple text match fails. Every example is taken from the source list and the Medicare catalogue snapshot.

Why a wrong flag matters

WHY IT MATTERS

The preferred marker feeds patient cost-sharing and prescriber-facing tools. A drug wrongly marked preferred can steer a prescriber toward a medicine the committee did not endorse. A drug wrongly left unmarked quietly removes a cheaper option from the patient. Neither failure announces itself, which is why the system is built to fail closed rather than to guess.

The objective, in one sentence

For each of the 1,355 entries on the preferred list, find every record in the plan's seven formularies that represents the same medicine, under any brand, salt, strength, or container, and mark exactly those records as preferred, adding nothing that is not already covered.

CHAPTER 2

The Journey of Three Drugs

The rest of this guide follows three real records. Each was chosen because it breaks the pipeline in a different way.

A single example would make the system look simpler than it is. These three between them exercise every branch of the matching logic.

The record	Why it was chosen
Insulin aspart Single ingredient, complicated naming	One molecule sold under several brands, in vials, pens, and cartridges, and also inside a premixed product. Sixteen source rows. Tests grouping and formulation handling.
Sulfamethoxazole-trimethoprim Two ingredients in one tablet	A combination antibiotic whose brand names do not appear in the catalogue at all. Tests brand-to-generic resolution and multi-ingredient matching.
Zoster vaccine live Vaccine with a misleading class	A shingles vaccine filed under an unhelpful class, sitting next to a different shingles vaccine that must not be matched to it. Tests ingredient-over-class precedence.

Table 2.1 The three worked examples. All three appear in both the source list and the Medicare catalogue snapshot.

What happens to insulin aspart, at a glance

Step	What the record looks like	Kind
1 Source text	insulin aspart (NOVOLOG) injection VIAL 100 unit/mL (10 mL)	Observed
2 Extracted identity	name: insulin aspart (novolog) / class: rapid-acting insulin analogues	AI
3 Grouped with 15 siblings	Pens, vials, cartridges and brands collapse toward shared drug records	Code
4 Checkpoint row	One row carrying the name, the class, and every source variant	Code
5 Searched in 7 plans	Each plan catalogue queried at the same time for lookalikes	Search
6 Shortlist kept	Only the closest similarity tiers survive	Code
7 Evidence packet	The shortlist becomes the model's entire visible world	Code
8 Same-drug judgment	Model keeps Insulin Aspart, Insulin Aspart FlexPen, NovoLOG	AI
9 Intersection guard	Anything not present in the packet is discarded	Code
1 Names resolved 0	MCR03212 becomes the readable name Insulin Aspart	Code
1 Pharmacist review 1	A report row pairs the preferred drug with its matches	Human
1 Database reconciled 2	Old flags cleared, verified rows marked preferred	Code

Table 2.2 The twelve steps, followed in detail from Chapter 3 onward. Steps marked AI are language model calls. Every AI step is followed by a deterministic step.

SAFETY CONTROL

Notice the ordering in steps 8 and 9. The model proposes, then code disposes. The model never writes to the database, and never introduces a record that was not already retrieved from the plan's own catalogue.

CHAPTER 3

Read the Source List

The pipeline begins with one column of human-written text, and nothing else.

The clinical committee delivers its recommendations as a spreadsheet with a single meaningful column. There is no drug code, no identifier, and no link to any catalogue. There are 1,355 populated rows and no blanks.

INPUT

One spreadsheet column of free text. Observed: 1,355 rows, no empty values. Everything downstream is derived from these strings.

What the strings actually contain

Source string (verbatim)	What is embedded in it
insulin aspart (NOVOLOG) injection VIAL 100 unit/mL (10 mL)	Generic name, brand in brackets, route, container, strength, volume
insulin aspart (ASPART) injection PEN 100 unit/mL (3 mL)	Same molecule, different container, brand slot filled with the generic
(Pres360) insulin aspart (FIASP) injection PEN 100 unit/mL (3 mL)	A programme prefix the pharmacy team uses internally
insulin asp prt-insulin aspart (NOVOLOG MIX 70/30) BRAND ONLY DAW 100 unit/mL vial soln	Abbreviated ingredient, a premix ratio, and a dispensing instruction
sulfamethoxazole-trimethoprim (BACTRIM;SEPTRA) tablet 400-80 mg	Two ingredients, two brands separated by a semicolon, paired strengths
zoster vaccine live (PF) (ZOSTAVAX) injection	A vaccine, a preservative-free marker, a brand, no strength at all
ACCU-CHEK AVIVA glucose monitoring kit	Not a medicine. A device, in capitals, with no generic name to parse
insulin aspart (NIACINAMIDE) injeciton VIAL 100 unit/mL (10 mL)	A spelling error in the source, preserved exactly as written

Table 3.1 Eight real rows from the source list, unedited. Note the typo in the final row: the pipeline has to tolerate it.

Terms hiding inside those strings

PLAIN-LANGUAGE DEFINITION

Generic name. The name of the active ingredient itself, such as insulin aspart. **Brand name.** A manufacturer's trade name for that ingredient, such as NovoLog or Fiasp. **Formulation.** The physical form the drug is supplied in, such as a vial, a pen, or a cartridge. **Strength.** How much active ingredient is present, such as 100 units per millilitre. One ingredient can appear under several brands, in several formulations, at several strengths, which is why one drug produces many rows.

Why this cannot be parsed with rules

A regular expression can strip the strength off the end of a string. It cannot tell you that insulin aspart is a rapid-acting insulin analogue, that Fiasp and NovoLog are the same molecule, or that an Accu-Chek Aviva is a glucose meter rather than a medicine. That is domain knowledge, and it is why the next stage uses a language model.

LIMITATION

The pipeline reads the source column and drops empty values. It does not validate the text beyond that. Malformed or ambiguous entries are passed to the extraction stage and handled there, or they fail there and are logged.

CHAPTER 4

Read the Drug Name and Class

A language model converts each free-text line into two clean fields, under tight constraints.

Why a model instead of a parser

The task is not to split a string. It is to answer two questions that require pharmacological knowledge: what is this drug called, and what kind of drug is it. The second question has no answer anywhere in the input text. It has to be supplied from knowledge of medicine.

INPUT

One source string, for example: insulin aspart (NOVOLOG) injection VIAL 100 unit/mL (10 mL)

The instruction the model is given

The model is not asked an open question. It is given a detailed protocol with worked examples covering medicines, combination products, supplements, vaccines and devices, and it is told to return nothing but JSON. Two instructions do most of the work:

- **Be specific, never broad.** The prompt explicitly forbids generic answers. It demands a subclass where one exists, so a beta blocker must be identified as cardioselective rather than simply as a beta blocker.
- **Handle devices as devices.** A separate branch tells the model to classify equipment by function, and to state whether it is manual or continuous, so a lancing device is never treated as a medicine.

The complete prompt is reproduced in Appendix B.

What comes back

```
{
  "drug_name": "insulin aspart (novolog)",
  "drug_class": "rapid-acting insulin analogues"
}
```

Figure 4.1 Illustrative output shape for the insulin aspart row. Both fields are lowercased by the pipeline before storage.

OUTPUT

Three columns per source row: the original text, the extracted name, and the extracted class. Written to a CSV checkpoint.

The controls around the call

Control	What it does
Temperature 0	Configured on the model client. This reduces variability between runs. It is not a guarantee of identical output, and the guide does not treat it as one.
Strict JSON parsing	The response is parsed as JSON. A response that will not parse raises an error for that row rather than being partially accepted.
Bounded concurrency	Ten rows are processed at a time behind a semaphore, with a pause between calls, so the API is used quickly without exceeding its limits.
Row isolation	A failure is caught per row and recorded with its error. One bad row does not end the run, and the failed rows are logged together at the end.
Restart cache	Results are written to a CSV. On a later run, if that file exists and a restart was not requested, the whole stage is skipped and the file is loaded instead.

Table 4.1 Reliability controls on the extraction stage, all read from the implementation.

LIMITATION

Temperature 0 lowers variability but does not make the model deterministic. Two runs over the same list can differ, particularly on ambiguous entries. The cache reduces exposure to this by making the first successful answer the one that is reused.

CHAPTER 5

Merge Doses of the Same Drug

Sixteen rows describing one insulin become a much smaller set of clinical decisions.

Why grouping happens before searching

WHY IT MATTERS

Searching and judging are the expensive parts of the pipeline. Doing them once per strength, when every strength of a drug will reach the same clinical conclusion, multiplies the cost and creates a way for two strengths of the same medicine to receive contradictory answers.

The grouping key

Rows are grouped on two fields together: the extracted drug name and the extracted class. Every original source string in a group is retained in a list, so nothing is lost. Each group is given an identifier.

```
group by ( preferred_primary_drug_name , preferred_drug_class )
keep      every original source string as preferred_drug_variants
assign    preferred_row_id
```

Before and after

Before: six of the sixteen insulin aspart source rows	
	insulin aspart (NOVOLOG) injection VIAL 100 unit/mL (10 mL)
	insulin aspart (NOVOLOG FLEXPEN) injection PEN 100 unit/mL (3 mL)
	insulin aspart (ASPART) injection PEN 100 unit/mL (3 mL)
	insulin aspart (ASPART) injection VIAL 100 unit/mL (10 mL)
	insulin aspart (FIASP) injection VIAL 100 unit/mL (10 mL)
	insulin aspart Non-Brand DAW injection 100 unit/ml crtg
After	Result
One drug record	insulin aspart, rapid-acting insulin analogues, carrying all of its source variants in a list

Table 5.1 Grouping in action. The source rows are observed. The exact number of resulting groups depends on how the extraction stage labelled each row, and is not asserted here.

Why one molecule can still occupy several groups

The key includes the class, not just the name. If the extraction stage gives two rows slightly different class labels, they form separate groups even though the molecule is the same. This is a deliberate trade. Splitting costs an extra search. Merging two genuinely different clinical categories under one name would be a clinical error.

The premixed products illustrate the same point from the other direction. `insulin asp prt-insulin aspart (NOVOLOG MIX 70/30)` contains insulin aspart, but it is a different clinical product from plain insulin aspart and belongs in its own group.

LIMITATION

Grouping quality is inherited from extraction quality. If the model labelled two identical drugs with different class strings, this stage cannot detect that and will produce two groups.

CHAPTER 6

One Row Per Drug

A small, stable contract that everything downstream depends on.

The grouped result is written out as a checkpoint. It is the boundary between understanding the source list and searching the catalogues.

Field	Meaning
preferred_row_id	Identifier for this group within this generated file
preferred_primary_drug_name	The clean drug name from extraction
preferred_drug_class	The therapeutic class from extraction
preferred_drug_variants	The list of every original source string in the group

Table 6.1 The four-field checkpoint contract.

Three examples

Drug name	Class	Variants carried
insulin aspart (novolog)	rapid-acting insulin analogues	16 source rows spanning vials, pens and cartridges
sulfamethoxazole-trimethoprim (bactrim;sepra)	sulfonamide antibacterial combinations	2 source rows, standard strength and double strength
zoster vaccine live (zostavax)	live attenuated viral vaccines	1 source row, no strength given

Table 6.2 The three worked examples as checkpoint rows. Names and variant counts are observed from the source list. Class strings are illustrative, since they are produced by the model at run time.

LIMITATION

The identifier is assigned from the row position in the generated file. It is stable while that file is in use. It is not a permanent key, and regenerating the checkpoint can assign a different number to the same drug. Nothing outside a single run should treat it as an identity.

CHAPTER 7

Search All Seven Formularies

Every drug record is searched against every plan catalogue at the same time.

Seven plans, seven catalogues

Formulary identifier	Plan
23379	PHP Medicare
24769	PHP Medicaid
23514	PHP Exchange Metal Level
24770	PHP Commercial
24771	PHP Commercial FEHB
23515	PHP Clear Metal Level
24772	PHP Intel Connected Care

Table 7.1 The seven configured formularies, read from the implementation. A drug can be preferred in one plan and absent from another, so all seven are searched independently.

The query

The drug name and the class are combined into one search string. The class is included only when it is known.

```
Drug: insulin aspart (novolog) | Therapeutic Class: rapid-acting insulin analogues

{
  "drug_class" : "Drug: insulin aspart (novolog) | Therapeutic Class: ...",
  "formulary_id": "23379"
}
```

Figure 7.1 The request sent to each of the seven catalogues. The same query goes to all seven, one call per plan.

Recall, in plain language

PLAIN-LANGUAGE DEFINITION

Recall. Casting a wide net on purpose. This stage is not trying to be right. It is trying not to miss anything. A wrong candidate here is harmless because the next stages will reject it. A missed candidate here is gone for good, because nothing downstream can retrieve it.

How the fan-out is protected

Control	Behaviour
Concurrency	The seven plan queries run in parallel rather than one after another, in a thread pool.
Authentication	Each request carries a bearer token resolved at run time from the platform secret store.
Timeout	Requests are given a fixed timeout so a hung plan cannot stall the run.
Retries	A failed request is retried a small number of times with a fixed wait between attempts.
Plan isolation	A plan that fails after its retries is logged and skipped. The other six plans still return results.
Pacing	A short delay is inserted before calls to avoid tripping rate limits.

Table 7.2 Protections on the fan-out search.

LIMITATION

Plan isolation keeps the run alive but changes the meaning of the result. If one plan's catalogue fails, the run completes with matches from six plans and no error is raised at the top level. The failure appears in the logs.

CHAPTER 8

The Paired Data Stores

One store is for finding things that are similar. The other is for holding the truth.

PLAIN-LANGUAGE DEFINITION

Embedding. A list of numbers that represents the meaning of a piece of text. Texts with similar meaning get similar lists. In this system each catalogue record is represented by 1,536 numbers, which is an observed property of the Medicare snapshot. **Vector search.** Finding the records whose lists of numbers are closest to the query's list. It can match records that share no words at all, which ordinary text search cannot do.

Why two stores rather than one

Store	What it holds and answers	Nature
The search store	Holds an embedding for each catalogue record plus a short text description. Answers the question: which records look like this one.	Approximate, ranked by similarity
The record store	Holds the full catalogue document for each record, including coverage details and the preferred marker. Answers: what exactly is this record.	Exact, addressed by identifier

Table 8.1 The division of responsibility. Search finds candidates. The record store is what actually gets updated at the end.

The shared key

Both stores carry the same identifier for the same record. That identifier is what allows a similarity hit to be turned into a real database update.

```

search store  formulary_row_id : "MCR03212"
               page_content    : "Drug: Insulin Aspart | Therapeutic Category:
                               Blood Glucose Regulators | Therapeutic
                               Class: Insulins | ..."
               dense_vector     : [ 1,536 numbers ]

record store  formulary_row_id : "MCR03212"
               drug_name        : "Insulin Aspart"
               preferred_drug    : false          <- the field this pipeline sets

```

Figure 8.1 The same insulin aspart record in both stores. The search content and identifier are observed from the Medicare snapshot. The record store fields are shown in the shape the pipeline reads and writes.

What the snapshot actually shows

Measure	Value	Basis
Catalogue records in the Medicare snapshot	4,579	Observed
Distinct drug names within those records	1,795	Observed
Numbers per embedding	1,536	Observed
Formularies configured in the pipeline	7	Observed from code
Records across all seven formularies	Not measured here	Only the Medicare catalogue was available for this guide

Table 8.2 Counts for the one catalogue available. The other six plans were not inspected, so no total across all seven is claimed.

LIMITATION

This guide was written against a single formulary export, the Medicare catalogue. Statements about record counts apply to that snapshot only. The catalogue is also a point-in-time copy, so identifiers within it should not be assumed stable across regenerated exports.

CHAPTER 9

Keep the Closest Matches

The shortlist is cut by similarity tier, not by row count.

Tiers rather than rows

The obvious way to shorten a ranked list is to keep the top few rows. This pipeline does something different. It collects the distinct similarity scores, takes the highest three of those, and then keeps every record whose score is one of them.

WHY IT MATTERS

Records that are equally good get equal treatment. If four containers of the same insulin score identically, keeping the top three rows would cut one of them arbitrarily. Keeping the top three scores keeps all four.

Worked example

Identifier	Catalogue name	Score	Tier	Outcome
MCR03212	Insulin Aspart	0.94	Tier 1	Kept
MCR03213	Insulin Aspart FlexPen	0.94	Tier 1	Kept
MCR01061	NovoLOG	0.91	Tier 2	Kept
MCR01060	NovoLOG FlexPen	0.91	Tier 2	Kept
MCR04374	Insulin Aspart PenFill	0.88	Tier 3	Kept
MCR01058	NovoLOG Mix 70/30	0.81	Tier 4	Dropped

Table 9.1 Identifiers and catalogue names are observed from the Medicare snapshot. **The similarity scores are illustrative**, chosen to show how tiers group tied records. They are not measured values.

Two records share tier one and two share tier two. A top-three-rows rule would have returned three records. The tier rule returns five, because records that scored the same are not separated.

LIMITATION

The tier rule keeps tied records together. It does not guarantee that every container and strength of a drug survives. In the example above the premixed product falls outside the top three tiers and is dropped at this stage. That is correct here, since a premix is a different clinical product, but the mechanism is similarity, not clinical reasoning.

CHAPTER 10

Build the Closed Evidence Packet

The shortlist is turned into the only thing the model will be allowed to see.

PLAIN-LANGUAGE DEFINITION

Closed evidence packet. A fixed list of candidate records, formatted as a table and placed inside the model's instructions. Closed means the model is asked to choose from this list and nothing else. It is the difference between asking which of these five records match and asking which records in the database match.

What goes into the packet

Column	Purpose
Formulary Plan Name	Which of the seven plans this candidate came from
Formulary Row ID	The shared identifier, used later to update the record
Description	The catalogue text, including drug name and therapeutic class
Similarity Score	How close the search judged it to be

Table 10.1 The four columns assembled for each candidate.

These are rendered as a Markdown table and embedded in the prompt. A table is used rather than raw JSON because the model is being asked to compare rows against one criterion, and a table makes that comparison structural rather than something it has to reconstruct.

The packet, and what comes back

```

PACKET (what the model can see)
| Plan          | Row ID   | Description                                     | Score | |
|---|---|---|---|---|
| PHP Medicare | MCR03212 | Drug: Insulin Aspart | Class: Insulins| 0.94 |
| PHP Medicare | MCR03213 | Drug: Insulin Aspart FlexPen | ...   | 0.94 |
| PHP Medicare | MCR01061 | Drug: NovoLOG | Class: Insulins      | 0.91 |

MODEL RESPONSE (names only, no identifiers)
[ {"formulary_drug_name": "Insulin Aspart"},
  {"formulary_drug_name": "Insulin Aspart FlexPen"},
  {"formulary_drug_name": "NovoLOG"} ]

INTERSECTION (code, not the model)
for each candidate in packet:
    if candidate.drug_name in model_response:
        keep candidate.formulary_row_id
-> MCR03212, MCR03213, MCR01061

```

Figure 10.1 Illustrative scores, observed identifiers and names. The model returns names. Code turns names back into identifiers by looking them up in the packet.

SAFETY CONTROL

The model is never given the power to emit an identifier. It returns names, and code maps those names back onto the packet it was given. A name the model invents has nothing to map onto and disappears. This is the central safety property of the design and it is examined in Chapter 13.

CHAPTER 11

Rules for Same Drug

The clinical protocol the model is required to follow, and why each rule exists.

The model is not asked whether two drugs are similar. It is given a numbered procedure. The full text is in Appendix C.

The medicine path

Rule		What it does
1	Exact active ingredient	Reject anything that does not contain the same active ingredient. The prompt states explicitly that drugs in the same class with a different ingredient must be rejected.
2	Salt and brand resolution	Resolve brand names to their generic ingredient, and treat salt forms as the same ingredient.
3	Formulation handling	Accept all formulations of a matched ingredient, including extended release and immediate release.
4	Class as a secondary check	Class is used only to catch contradictions. It is explicitly subordinate to ingredient matching.

Table 11.1 The rule hierarchy for medicines, in priority order.

WHY IT MATTERS

Rule one exists because two drugs in the same class are not interchangeable. Two beta blockers are both beta blockers and are still different medicines. Without this rule, a similarity-driven system will happily treat class membership as equivalence, which is a clinical error.

The device path

Devices cannot be matched by ingredient because they have none. They are matched on function and compatibility instead: whether the device administers or stores the drug in question, whether the specification fits, and whether the device carries the drug’s brand name. A device whose route or physical form is incompatible is rejected.

The three examples, judged

Preferred drug	Candidate	Judgment
insulin aspart	Insulin Aspart FlexPen	Match. Same ingredient, different container. Rule 3 accepts formulation differences.
insulin aspart	NovoLOG	Match. Rule 2 resolves the brand to insulin aspart.
insulin aspart	Insulin Lispro	Reject. Same class, both rapid-acting insulins, different ingredient. Rule 1 rejects it.
sulfamethoxazole-trimethoprim	Sulfamethoxazole-Trimethoprim	Match. Both ingredients present. The brands BACTRIM and SEPTRA never needed to appear in the catalogue.
sulfamethoxazole-trimethoprim	Polymyxin B-Trimethoprim	Reject. Shares trimethoprim but not sulfamethoxazole. A combination requires every ingredient, not one of them.
zoster vaccine live	Zostavax	Match. The same vaccine, even though the catalogue files it under the unhelpful class Non-FRF Drugs. Rule 4 keeps class subordinate, so the odd class does not block the match.
zoster vaccine live	Shingrix	Reject. Also a shingles vaccine, and filed under the class Vaccines, but it is recombinant rather than live. Different product, so rule 1 rejects it.

Table 11.2 All candidate names and their catalogue classes are observed in the Medicare snapshot. The judgments show how the rules apply.

WHY IT MATTERS

The last two rows are the reason class cannot be the primary key. Zostavax and Shingrix are both shingles vaccines. The catalogue puts them in different classes. Matching on class would pair the preferred drug with the wrong one and miss the right one. Matching on the product itself gets both answers right.

The response contract

The prompt requires a JSON array of names and forbids explanatory text entirely. An empty array is the correct answer when nothing matches.

```
[{"formulary_drug_name": "EXACT_NAME"}]      or      []
```

CHAPTER 12

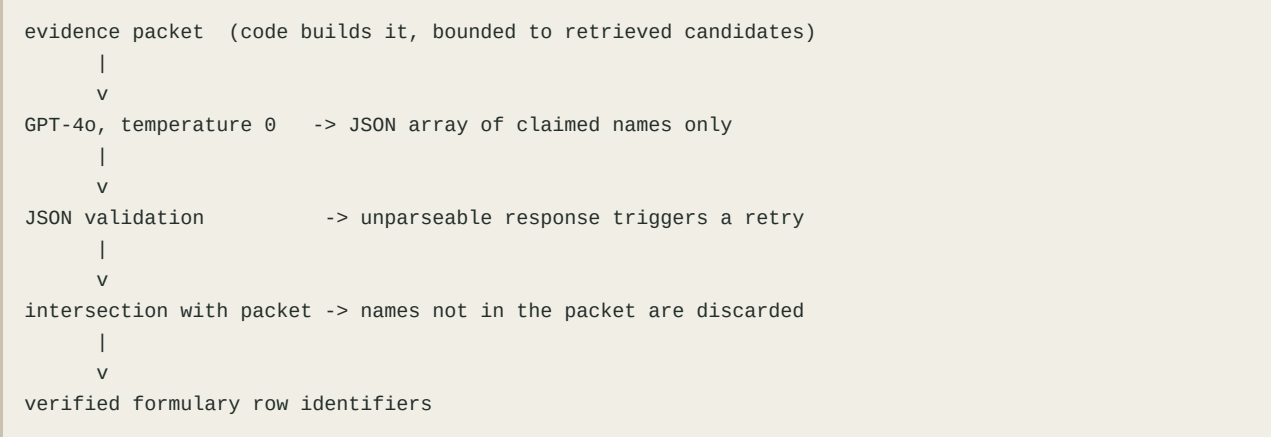
AI Clinical Judgment

What the model is trusted with, and what it is deliberately not trusted with.

The model decides	Whether a candidate in the packet is the same medicine as the preferred drug. It resolves brands to ingredients, accepts salt and formulation differences, and rejects same-class impostors.
The model does not decide	Which records exist. Which identifiers are real. What gets written to the database. Whether a run succeeded. All of these are handled by code.

Table 12.1 The trust boundary.

The chain



Reliability controls

Control	Behaviour
Response validation	The response is parsed before it is accepted. An unparseable answer is treated as a failure and retried.
Retry with backoff	Failed calls are retried up to five times, with the wait doubling each attempt and a small random jitter added so parallel workers do not retry in lockstep.
Row isolation	A drug that fails all its retries is recorded as failed with its error message. The run continues.
Chunking	The drug list is processed in sequential chunks of fifty.
Checkpoints	Each chunk writes its results, and any failed rows, to disk before the next chunk starts.
Bounded parallelism	Within a chunk, rows are processed by a small thread pool rather than all at once.

Table 12.2 Controls read from the implementation.

WHY IT MATTERS

Chunking with checkpoints is what makes a long run survivable. A failure in the sixth chunk of ten leaves the first five chunks on disk. Without it, a single late failure would discard hours of model calls that had already been paid for.

CHAPTER 13

Block Unsupported Matches

The one guarantee the system makes, and the mechanism that enforces it.

The five gates

Gate		Effect
1	Bounded retrieval	Candidates can only come from the plan's own catalogues. Nothing else is ever retrieved.
2	Closed packet	The model sees only those candidates, formatted as a table.
3	Names only	The model returns names. It is never asked for, and cannot supply, a database identifier.
4	Intersection	Code walks the packet and keeps a candidate only if its name appears in the model's response.
5	Identifier from the packet	The identifier written downstream is read from the packet record, never from the model's text.

Table 13.1 The safety chain. Gates 4 and 5 are the ones that make a fabricated answer harmless.

A fabricated claim, removed

PACKET

MCR03212 Insulin Aspart
MCR03213 Insulin Aspart FlexPen

MODEL RESPONSE

[{"formulary_drug_name": "Insulin Aspart"},
 {"formulary_drug_name": "Insulin Aspart FlexPen"},
 {"formulary_drug_name": "Insulin Aspart SuperPen"}] <- not in the packet

AFTER INTERSECTION

MCR03212, MCR03213

The third name matched no packet record, so it produced no identifier and reached nothing downstream. No error was needed to stop it.

Figure 13.1 Illustrative fabrication. The mechanism does not detect the invention. It simply gives it nowhere to go.

SAFETY CONTROL

The **local invariant**. Every identifier emitted by this stage was read from a record that the plan's own catalogue returned for this drug. The model can narrow that set. It cannot extend it. This is why a drug cannot be marked preferred unless it is already covered.

Empty is not the same as failed

Outcome	Meaning
Successful empty result	The model examined the packet and matched nothing. An empty array is a valid answer. The drug simply has no preferred match in that catalogue.
Failed judgment	The call could not be completed or parsed after its retries. The row is recorded as failed with its error message and appears in the failure output.

Table 13.2 These two look similar in a results file and mean opposite things. The pipeline separates them into different outputs.

CHAPTER 14

Resolve Real Drug Names

Identifiers are correct but unreadable. Before a human sees them, they become names again.

The verified result is a list of identifiers. No reviewer can check MCR03212 by eye. Each identifier is looked up in the record store and replaced with the drug name held there.

```
filter      { "formulary_row_id": "MCR03212" }
projection  { "drug_name": 1, "_id": 0 }
result      { "drug_name": "Insulin Aspart" }
```

Figure 14.1 The lookup. Only the name is requested, so the rest of the document is not transferred.

How the report is assembled

- Lookups for a row run concurrently rather than one at a time.
- Names are deduplicated, so two identifiers resolving to the same name produce one line.
- Matched formulary drugs are joined with a triangle marker, written in the source as the escape `\u25B6`.
- The original preferred variants are joined with a bullet marker, written as `\u2022`.

The two different markers let a reviewer tell at a glance which side of the comparison a line belongs to: bullets are what the committee asked for, triangles are what the pipeline found.

LIMITATION

Deduplication is by name, so the count of names in the report can be smaller than the count of identifiers behind it. The set is unordered, so line order is not stable between runs. An identifier with no matching record resolves to nothing and is silently dropped from the readable list, which means the report can under-report what the database update will touch.

CHAPTER 15

Pharmacist Review

The artifact a human reads, the four things they check, and what the review cannot do.

Column	What the reviewer sees
preferred_row_id	Which drug group this row describes
preferred_primary_drug_name	The extracted drug name
preferred_drug_class	The extracted class
preferred_drug_variants	Every original source string, bulleted
formulary_row_ids	The verified identifiers
formulary_drugs_found_to_preferred	The resolved catalogue names, marked with triangles

Table 15.1 The six-column review contract.

One row, as a reviewer sees it

preferred drug	insulin aspart (novolog)
class	rapid-acting insulin analogues
variants	• insulin aspart (NOVOLOG) injection VIAL 100 unit/mL (10 mL) • insulin aspart (ASPART) injection PEN 100 unit/mL (3 mL) • insulin aspart (FIASP) injection VIAL 100 unit/mL (10 mL)
matched drugs	■ Insulin Aspart ■ Insulin Aspart FlexPen ■ NovoLOG

Figure 15.1 Source variants and catalogue names are observed. The layout follows the markers the code writes.

The four checks

Check	What the reviewer is looking for
Ingredient	Is every matched drug genuinely the same active ingredient? A different insulin appearing here is the failure this check catches.
Formulation	Are the containers and strengths sensible, and is anything obviously missing from the family?
Plan spread	Do the matches cover the plans they should? A drug matching in one plan only may indicate a failed catalogue query rather than a genuine absence.
Unresolved gap	Does the number of identifiers exceed the number of names shown? That difference points at records that could not be resolved.

Table 15.2 The four inspections the report supports.

Why review is not optional: the device case

Medicines behave reasonably well because ingredient matching is a strong signal. Devices do not. A glucose monitoring kit such as ACCU-CHEK AVIVA glucose monitoring kit has no active ingredient at all, so rules one to three of the matching protocol do not apply to it. Matching falls back to functional compatibility and shared naming, which is a much looser test.

A search for a monitoring kit can therefore surface a mixed family: lancets, test strips, lancing devices, and meters, all sharing a brand and all plausibly compatible, but not all the same product. Whether that family is correct is a judgment a pharmacist has to make. It is not a failure of the pipeline so much as the point at which the pipeline stops being able to decide.

NOTE

This device scenario is presented as an illustration of the risk, not as an observed result. The review output for the Accu-Chek entries was not available when this guide was written, and the Medicare catalogue snapshot examined here contains no Accu-Chek records at all, so the matches would have come from other plan catalogues that were not inspected.

LIMITATION

The review report carries no confidence score and no per-match plan name, so a reviewer cannot see how close a match was or which plan each match came from without going back to the intermediate files. More importantly, the orchestration script writes the report and then proceeds directly to the database update. Nothing pauses for approval. The review is an artifact a human can read, not a gate the pipeline waits on.

CHAPTER 16

Clear Old Flags and Set New Ones

The database is reconciled to the verified result rather than patched.

PLAIN-LANGUAGE DEFINITION

Desired-state reconciliation. Instead of working out what changed since last time and applying the difference, the system declares what the world should look like now and makes it so. Here that means clearing the preferred marker everywhere, then setting it only on the verified records.

Why reset before mark

A drug that was preferred last cycle and is not preferred now has to lose its marker. An incremental update would need logic to work out which markers to remove, and that logic is exactly where stale flags survive. Clearing everything first makes removal automatic: anything not in the new verified set simply never gets marked again.

The two operations

```
STEP 1 reset, matched on the document key
update_many(
  { "_id": { "$in": [ ...batch of document keys... ] } },
  { "$set": { "preferred_drug": false } } )

STEP 2 mark, matched on the formulary row identifier
UpdateOne(
  { "formulary_row_id": "MCR03212" },
  { "$set": { "preferred_drug": true } } )
... issued as an unordered bulk write
```

Figure 16.1 The reset and the mark match on different fields. The reset walks the collection by document key. The mark addresses records by their formulary row identifier.

Preparing the identifiers

Verified identifiers arrive as lists that may have been written to and read back from CSV, so they can be real lists or strings that look like lists. They are parsed defensively, flattened, and deduplicated into a single set before any write happens.

How the writes are protected

Control	Behaviour
Batching	Both phases work in batches of one hundred documents.
Parallel reset	The reset runs its batches through a small thread pool while paging through the collection.
Retries	Batches are retried up to three times on transient database errors, with an exponential wait.
Unordered writes	The mark phase issues its updates unordered, so one failing update does not stop the rest of the batch.

LIMITATION

These operations are not transactional. The reset and the mark are separate sets of writes with no surrounding transaction. Between them, and during them, the collection is in a partially updated state where records that should be preferred are not yet marked. Anything reading the collection at that moment sees an incomplete answer.

LIMITATION

A batch that exhausts its retries is logged and the loop continues. The run reports completion. The result is a collection where some records were reset but never re-marked, and nothing in the current implementation halts the run or raises this to the caller.

SAFETY CONTROL

After a fully successful run, the set of records marked preferred is exactly the set of verified identifiers, and every one of those came from a candidate the plan's own catalogue returned. The qualifier matters: this holds when every batch in both phases succeeded.

CHAPTER 17

The Live Formulary Record

What the pipeline actually changes, and what it leaves alone.

The record store holds one document per catalogue line. The pipeline reads two fields from it and writes exactly one.

Group	Fields	What the pipeline does
Identity	formulary_row_id, drug_name	Read. The identifier is the join key, the name is used for the review report.
Clinical	therapeutic category and class	Not touched by this pipeline. Present in the search text.
Coverage	Plan and tier information	Not touched.
Codes	Product and packaging identifiers	Not touched.
State	preferred_drug	Written. This single boolean is the pipeline's entire output into the live system.

Table 17.1 Field groups in a catalogue document. Only the state field is modified.

The division of labour, restated

The search store finds candidates and is never written to by this pipeline. The record store is the system of record and receives the one field that changes. Keeping these separate is what allows the search index to be rebuilt or retuned without any risk to the coverage data.

LIMITATION

The document carries the current value of the marker and nothing about how it was reached. There is no run identifier, no timestamp, no record of which preferred drug caused it, and no note of whether a model or a human confirmed it. Reconstructing why a particular record is marked requires the intermediate files from that run.

LIMITATION

This guide describes the local snapshot that was available for inspection, and the collection the code is configured to write to. It does not assert what other applications read that collection. Chapter 1 describes prescriber-facing and cost-sharing uses as the business motivation for the field, which is what the marker is for, not a claim proven by this repository.

CHAPTER 18

End to End Summary

The whole journey on one spread, and what the design teaches.

Step	What happened to insulin aspart	Kind
Source	insulin aspart (NOVOLOG) injection VIAL 100 unit/mL (10 mL), one of sixteen insulin aspart lines	Observed
Identity	name and therapeutic class extracted by the model	AI
Group	sixteen source lines collapse toward shared drug records	Code
Checkpoint	one row carrying name, class, and all variants	Code
Search	seven plan catalogues queried in parallel	Search
Shortlist	top three similarity tiers kept, ties preserved	Code
Packet	shortlist becomes the model's entire visible world	Code
Judgment	Insulin Aspart, Insulin Aspart FlexPen and NovoLOG accepted; Insulin Lispro rejected	AI
Guard	names not present in the packet discarded	Code
Resolve	MCR03212 becomes Insulin Aspart	Code
Review	one report row pairing variants with matches	Human
Reconcile	markers cleared, then verified records marked	Code

Table 18.1 The full journey. Two AI steps, eight deterministic steps, one search step, one human step.

Who decides what

Actor	Authority
The model	Whether a candidate is the same medicine. Nothing else.
Deterministic code	What is retrieved, what the model may see, which claims survive, which identifiers are used, and what is written.
The pharmacist	Whether the result is clinically sensible. Advisory in the current implementation.
The record store	The single preferred marker per catalogue line.

Five lessons

Lesson		Why it held up
1	Retrieve broadly, then reason narrowly	Search is tuned for recall because a missed candidate is unrecoverable. Judgment is applied afterward, on a small set, where being strict is cheap.
2	Bound what the model can see	A closed evidence packet turns an open-ended question into a multiple choice one. Most of the reliability of this system comes from that single decision.
3	Let the model propose, never dispose	Returning names rather than identifiers, then intersecting with the packet, makes fabrication structurally inert instead of something to be detected.
4	Name the failure mode in the prompt	Stating that same-class drugs with different ingredients must be rejected does more than asking for accuracy. Specific prohibitions beat general exhortation.
5	Treat a long AI run as a distributed system	Chunking, checkpoints, backoff with jitter and per-row isolation are what let the pipeline finish against thousands of records and a fallible API.

NOTE
The honest closing note: the strongest guarantee this system makes is narrow. It guarantees that nothing is marked preferred unless the plan's own catalogue returned it. It does not guarantee that every correct match was found, that the database is consistent mid-run, or that a human approved the result. Those are the boundaries, and Appendix E lists what would move them.

Appendix A. Glossary

Every term used in this guide, defined once.

Term	Definition
Active ingredient	The substance in a medicine that produces the effect. The basis of matching for medicines.
Backoff with jitter	Waiting longer after each failed retry, plus a small random amount so parallel workers do not retry in unison.
Brand name	A manufacturer's trade name for a drug, such as NovoLog.
Checkpoint	A file written partway through a run so that work already done survives a later failure.
Closed evidence packet	A fixed candidate list given to the model, which it may choose from but not extend.
Combination drug	A single product containing more than one active ingredient. Every ingredient must match.
Embedding	A list of numbers representing the meaning of a text. Similar meanings produce similar lists.
Formulary	The list of drugs an insurance plan covers.
Formulation	The physical form of a drug, such as a vial, pen or tablet.
Generic name	The name of the active ingredient itself.
Idempotent	An operation that produces the same end state whether it runs once or several times.
Preferred Drug List	The clinical committee's list of drugs to reach for first.
Recall	Retrieving broadly so that nothing is missed, accepting some wrong candidates.
Reconciliation	Making the database match a declared desired state rather than applying a difference.
Salt form	A chemical variant of an ingredient that is treated as the same ingredient for matching.
Semantic search	Finding records by meaning rather than by shared words.
Similarity tier	A distinct similarity score. Records sharing a score share a tier and are kept or dropped together.
Temperature 0	A model setting that reduces variability between runs. It is not a determinism guarantee.
Vector search	Finding records whose embeddings are closest to the query's embedding.

Appendix B. The extraction prompt

The complete instruction used in Chapter 4, reproduced from the implementation.

From the Preferred drug string: "{drug_string}", extract the following information with utmost precision:

1. Primary Name Extraction

- Identify the main name along with any brand name present.
- Example: For "losartan-hydrochlorothiazide (HYZAAR) tablet 100-12.5 mg", the primary name is "losartan-hydrochlorothiazide (HYZAAR)".

2. Precise Class Identification (No Broad Classes Allowed)

For Drugs:

- Identify the most specific drug class possible.
- Always include any available subclass, mechanism, or specific detail. For example, return "Beta-1 Selective Beta-Blockers (Cardioselective Beta-Blockers)" instead of simply "Beta-Blockers."
- If an acronym is used, provide the full form followed by the acronym in brackets.
- For combination drugs, list each precise class separated by a comma.

For Supplements:

- Identify the supplement category with granular specificity, such as "Fat-Soluble Vitamins (Vitamin D)", "Water-Soluble Vitamins (Vitamin C)", "Omega-3 Fatty Acids (Fish Oil Supplements)".

For Vaccines:

- Identify the vaccine type with the most precise classification possible.
- Include additional details such as "Inactivated Influenza Vaccines", "Live Attenuated Influenza Vaccines", "Recombinant Influenza Vaccines", or any adjuvanted formulations, rather than simply "Vaccines".

For Devices:

- Determine the device's precise functional class, such as "Insulin Delivery Devices", "Glucose Monitoring Devices", "Lancets and Lancing Devices".
- Explicitly specify whether the device is Manual or Continuous, for example "Insulin Delivery Devices (Manual)".

Important: If any additional detail is available, such as "central" in "alpha-2 adrenergic agonists (central)", it must be included. Do not default to a broad category if a more precise subclass is derivable from the input.

3. Handling Edge Cases

- Resolve any ambiguities by providing the most specific class possible.
- For combination products, ensure every component is classified precisely.
- Always include additional qualifiers such as subtype, mechanism, or operation mode for devices.

Examples

```
"acebutolol (SECTRAL) capsule 200 mg"
  drug_name : "acebutolol (SECTRAL)"
  drug_class : "Beta-1 Selective Beta-Blockers (Cardioselective Beta-Blockers)
               with Intrinsic Sympathomimetic Activity (ISA)"
"atenolol (TENORMIN) tablet 50 mg"
  drug_name : "atenolol (TENORMIN)"
  drug_class : "Beta-1 Selective Beta-Blockers (Cardioselective Beta-Blockers)"
"diltiazem (CARDIZEM CD) capsule 120 mg"
  drug_name : "diltiazem (CARDIZEM CD)"
  drug_class : "Non-Dihydropyridine Calcium Channel Blockers (Benzothiazepines)"
"losartan-hydrochlorothiazide (HYZAAR) tablet 100-12.5 mg"
```

```
    drug_name : "losartan-hydrochlorothiazide (HYZAAR)"
    drug_class : "Angiotensin II Receptor Blockers (ARBs), Thiazide Diuretics"
"bd insulin pen needle uf mini"
    drug_name : "bd insulin pen needle uf mini"
    drug_class : "Insulin Delivery Devices (Manual)"
"accu-chek aviva"
    drug_name : "accu-chek aviva"
    drug_class : "Glucose Monitoring Devices (Manual)"
"accu-chek softclix lancets"
    drug_name : "accu-chek softclix lancets"
    drug_class : "Lancets and Lancing Devices (Manual)"
"vitamin D3 (cholecalciferol) supplement 1000 IU"
    drug_name : "vitamin D3 (cholecalciferol)"
    drug_class : "Fat-Soluble Vitamins (Vitamin D)"
"omega-3 fish oil softgels"
    drug_name : "omega-3 fish oil softgels"
    drug_class : "Omega-3 Fatty Acids (Fish Oil Supplements)"
"influenza vaccine"
    drug_name : "influenza vaccine"
    drug_class : "Inactivated Influenza Vaccines"
```

Final Output Format

Return the result as a JSON object in the following format:

```
{
  "drug_name" : "<primary name>",
  "drug_class" : "<most precise class>"
}
```

Ensure that the output always includes the most precise and detailed drug class, avoiding any broad or generic classifications.

Appendix C. The matching prompt

The complete instruction used in Chapters 11 and 12, reproduced from the implementation.

PHARMACOLOGICAL AND DEVICE MATCHING PROTOCOL

1. Categorization Phase

Classify each entry as either:

- Drug/Medication: if the 'Drug: ' field contains a chemical or generic name
- Device: if the 'Drug: ' field describes equipment, for example "Diskhaler", "Lancing Device"

2. Drug Processing

a. Active Ingredient Supremacy

- Eliminate drugs NOT containing the EXACT ACTIVE INGREDIENT of {preferred_drug_name}
IMPORTANT: Reject same-class drugs with different active ingredients
- Match salt forms, for example "X Phosphate" and "X"
- Brand names MUST be resolved to their generic equivalents using standard pharmacological references. Example: 'Bydureon' to exenatide, 'Byetta' to exenatide
- Match ALL brand variants of the active ingredient, for example 'Bydureon BCise' equals exenatide

b. Formulation Handling

- Include ALL formulations, IR, XR and others, of the matched active ingredient
- Example: "Oseltamivir Suspension" matches "Oseltamivir"

c. Class Validation

- Confirm remaining drugs belong to {preferred_drug_class}
- Class matching is SECONDARY to active ingredient matching. Only reject drugs if the class explicitly contradicts the preferred drug's class.

3. Device Processing

a. Functional Compatibility

- Match devices supporting administration or storage of {preferred_drug_name}
- Examples: insulin pens and insulin; spacers and inhalers
- Devices containing the preferred drug's BRAND NAME in their title are automatically compatible, for example 'Bydureon Pen-Injector' supports exenatide

b. Specification Matching

- Compare device type such as needle gauge or inhaler type, continuous versus manual devices, and brand compatibility
- Assume compatibility for devices sharing naming conventions with matched drugs

c. Exclusions

- Reject devices incompatible with {preferred_drug_name}'s administration route or physical properties

4. Final Output Rules

- Return BOTH matched drugs AND devices
- Format:
[{"formulary_drug_name": "EXACT_DRUG_OR_DEVICE_NAME"}]
- If no matches, return an empty array: []

- DO NOT INCLUDE ANY EXPLANATORY TEXT OR COMMENTS IN THE RESPONSE.

FORMULARY ENTRIES
{top_formulary_drugs_markdown_table}

Appendix D. Data counts and snapshot notes

What was measured, what was read from code, and what was not available.

Observed values

Measure	Value	How it was obtained
Preferred drug list rows	1,355	Counted from the source spreadsheet, after dropping empty values. No empties were present.
Insulin aspart source rows	16	Counted from the source spreadsheet.
Sulfamethoxazole-trimethoprim source rows	2	Counted from the source spreadsheet.
Zoster vaccine source rows	1	Counted from the source spreadsheet.
Medicare catalogue records	4,579	Counted from the Medicare export.
Distinct drug names in that catalogue	1,795	Parsed from the catalogue search text.
Embedding size	1,536	Measured directly on a catalogue record.

Read from the implementation

Setting	Value
Configured formularies	7
Extraction concurrency	10 at a time, with a pause between calls
Similarity tiers kept in the matching flow	3
Verification chunk size	50 drugs per chunk
Verification retries	up to 5, exponential backoff with jitter
Search retries	3 attempts, fixed wait, fixed timeout
Database batch size	100, for both reset and mark
Reset parallelism	4 workers

Not available when this guide was written

- Six of the seven formulary catalogues. Only the Medicare export was inspected, so no total record count across all seven plans is stated anywhere in this guide.
- Real similarity scores. Every score shown is illustrative and labelled as such.
- The generated checkpoint and review output files, so no group count or match count from a production run is asserted.
- The Accu-Chek review result, which is why Chapter 15 presents the device risk as an illustration rather than as an observed outcome.

NOTE

Every number in the main chapters is either labelled observed, labelled illustrative, or attributed to the implementation. Where a figure would have required data that was not available, the guide states that it was not measured rather than estimating it.

Appendix E. Known limitations

Collected from across the guide, with what would address each.

Limitation	Effect	What would address it
Temperature 0 is not determinism	Two runs can differ on ambiguous entries.	Treat the cached first answer as authoritative; review class strings that vary.
Grouping inherits extraction quality	Inconsistent class labels split one drug into several groups.	Normalise class strings, or group on drug name with class as a secondary attribute.
Row identifiers are snapshot-scoped	Regenerating the checkpoint can renumber drugs.	Derive a stable key from the drug name and class rather than row position.
Plan failures are silent at the top level	A run can complete using six of seven catalogues.	Surface per-plan success counts in the run summary and fail loudly on a missing plan.
Similarity tiers are not clinical logic	A container or strength can fall outside the kept tiers.	Review formulation coverage, or widen tiers for drugs with large product families.
Name deduplication hides identifiers	The review report can show fewer names than the update will touch.	Report the identifier count alongside the names, and flag unresolved identifiers explicitly.
Review is advisory, not a gate	The script writes the report and proceeds directly to the database update.	Split the run into two commands, or require an approval file before the sync stage executes.
Reconciliation is not transactional	The collection is inconsistent between reset and mark.	Write to a staging field and swap, or run the sync in a maintenance window.
Exhausted batches do not halt the run	Records can be reset and never re-marked, and the run still reports success.	Track failed batches and exit non-zero, or re-drive failures before completing.
No decision provenance on the record	A marked record carries no trace of why it was marked.	Store a run identifier and the source preferred drug alongside the marker.
Development sampling in the orchestrator	The main script samples a small number of rows when loading the source list, which would silently process a fraction of the list in a production run.	Remove the sampling before any full run.

Appendix F. Source references

Where each part of this guide came from.

Part of this guide	Source
Chapter 3	The preferred drug list spreadsheet
Chapter 4	The extraction module and the extraction prompt
Chapters 5 and 6	The condensation module
Chapters 7 and 9	The formulary search module
Chapter 8	The Medicare formulary export and the database configuration
Chapters 10, 11, 12 and 13	The intersection module and the matching prompt
Chapter 14	The name resolution utilities
Chapter 15	The report assembly utilities and the orchestration script
Chapters 16 and 17	The preferred flag synchronisation module
Model and platform settings	The language model configuration module

This guide was generated programmatically from a single reproducible source file so that it can be regenerated when the implementation changes.